Chain-Native Threshold Signatures: FROST and CGGMP21 on Zoo Network's T-Chain

Replacing External MPC with Consensus-Secured Threshold Cryptography

Version 2026.04

Zoo Kang

Zoo Labs Foundation

2026

Abstract

We present T-Chain (ThresholdVM), a chain-native threshold signature subsystem for Zoo Network that eliminates the need for external multi-party computation (MPC) services. T-Chain embeds FROST (Flexible Round-Optimized Schnorr Threshold signatures), CGGMP21 (threshold ECDSA with identifiable abort), and Linear Secret Sharing (LSS) directly into the validator set, so that validators *are* the MPC parties. All keygen, signing, and reshare operations execute as on-chain transactions secured by consensus, removing the operational burden of sidecar MPC deployments. We describe the architecture of ThresholdVM, the RPC surface (`threshold_keygen`, `threshold_sign`, `threshold_reshare`), the session management layer, and the security guarantees that follow from coupling threshold cryptography with stake-weighted consensus. We prove t-of-n unforgeability under the Discrete Logarithm assumption for FROST and under the Strong RSA and DDH assumptions for CGGMP21, and show how the LSS Composition Theorem allows FROST, CMP, TFHE, Ringtail, and BLS protocols to share a single access structure. Formal verification of all protocols is carried out in Lean 4 (see `Crypto/FROST.lean`, `Crypto/Threshold/CGGMP21.lean`, `Crypto/Threshold/LSS.lean`, `Crypto/Threshold/Composition.lean`). We report on the migration path from external MPC StatefulSets to T-Chain, including an HTTP compatibility layer for existing bridge integrations, and benchmark GPU-accelerated FHE operations within the threshold decryption pipeline.

Contents

1 Introduction **4**
- 1.1 Contributions . . . 4
- 1.2 Relation to Other Zoo Papers . . . 4

2 T-Chain Architecture **4**
- 2.1 ThresholdVM Overview . . . 4
- 2.2 RPC API . . . 5
- 2.3 Session Management . . . 5
- 2.4 Permissioned Key Creation . . . 5

3 FROST Protocol **6**
- 3.1 Overview . . . 6
- 3.2 Distributed Key Generation . . . 6
- 3.3 Two-Round Signing . . . 6

4 CGGMP21 Protocol **6**
- 4.1 Overview . . . 6
- 4.2 Key Properties . . . 7
- 4.3 Offline/Online Separation . . . 7

5 LSS Foundation **7**
- 5.1 Linear Secret Sharing . . . 7
- 5.2 The Composition Theorem . . . 7

6 MPC Wallet Creation **8**
- 6.1 Permissioned Keygen . . . 8
- 6.2 Keygen Flow . . . 8
- 6.3 Public Key Derivation . . . 8

7 Security Analysis **9**
7.1 Open DKG Safety 9
7.2 Consensus-Coupled Threshold Guarantees 9
7.3 Identifiable Abort 9

8 GPU Acceleration **9**
8.1 FHE Operations in T-Chain 9
8.2 Performance 10

9 Formal Verification **10**

10 Migration from External MPC **11**
10.1 Replacing the MPC StatefulSet 11
10.2 Bridge Integration 11
10.3 HTTP Compatibility Layer 11

11 Related Work **12**

12 Conclusion **12**

1 Introduction

Threshold signature schemes distribute signing authority across n parties such that any t can jointly produce a valid signature while fewer than t learn nothing about the secret key. Existing blockchain deployments rely on *external* MPC clusters—separate StatefulSets, dedicated key management services, or third-party custodians (Fireblocks, Lit Protocol)—to perform keygen and signing. This introduces three risks: (1) **operational risk**—MPC nodes must be deployed and rotated independently, with no consensus penalties for downtime; (2) **trust asymmetry**—the MPC quorum may differ from the validator quorum; (3) **latency overhead**—cross-service RPC adds round-trips to every signing request.

T-Chain eliminates all three by making validators the MPC parties. Consensus secures the MPC transcript: if a validator equivocates during a signing round, the same slashing conditions that penalize double-signing apply. The result is a chain-native threshold signature service with no additional trust assumptions beyond the consensus threshold.

1.1 Contributions

This paper makes the following contributions:

- A complete specification of ThresholdVM, a virtual machine that executes FROST, CG-GMP21, and LSS protocols as first-class chain operations.
- An RPC API for threshold key management: `threshold_keygen`, `threshold_sign`, `threshold_reshare`, `threshold_status`.
- Security proofs showing that chain-native MPC inherits consensus-level safety: an adversary must corrupt $\geq t$ validators *and* break consensus to forge a signature.
- The LSS Composition Theorem, demonstrating that FROST, CMP, TFHE, Ringtail, and BLS can share a single t-of-n access structure over the same secret shares.
- A migration guide from external MPC StatefulSets to T-Chain, including an HTTP compatibility layer for bridge contracts.
- Benchmarks of GPU-accelerated FHE operations within the threshold decryption pipeline.

1.2 Relation to Other Zoo Papers

T-Chain integrates with several components described in companion papers: the network topology and consensus layer [12], the staking and economic model [13], homomorphic encryption infrastructure [14], decentralized identity and DID management [15], and hierarchical key derivation [16]. For the underlying Lux-layer threshold primitives, see [17] and [18].

2 T-Chain Architecture

2.1 ThresholdVM Overview

ThresholdVM is a native chain on Zoo Network, registered at chain creation alongside the EVM L2, Identity Chain, and Experience Ledger. It exposes a minimal state machine with three transaction types:

1. **KeygenTx**: Initiates a distributed key generation session among a quorum of validators.
2. **SignTx**: Requests a threshold signature over an arbitrary message digest.
3. **ReshareTx**: Re-distributes key shares to a new validator set without changing the public key.

Each transaction type maps to a multi-round interactive protocol. ThresholdVM manages session state, enforces round deadlines, and commits the protocol transcript to the chain once all rounds complete or a timeout triggers abort.

2.2 RPC API

The T-Chain RPC surface is accessible at `/v1/threshold`:

Method	Description
`threshold_keygen`	Start a t-of-n DKG session; returns `session_id` and public key on completion
`threshold_sign`	Submit a message hash for threshold signing; returns partial signatures, then aggregated signature
`threshold_reshare`	Initiate share redistribution to a new committee
`threshold_status`	Query session state (pending, round k, complete, aborted)
`threshold_pubkey`	Retrieve the group public key for a given `key_id`

Table 1: T-Chain RPC methods

2.3 Session Management

A session $\mathcal{S} = (\texttt{session\_id}, \texttt{protocol}, t, n, \texttt{participants}, \texttt{round}, \texttt{deadline})$ tracks the lifecycle of each interactive protocol instance. Sessions are created by a `KeygenTx` or `SignTx` and progress through rounds as validators submit messages. If a validator fails to submit a round message before the deadline, the session enters *identifiable abort*: the non-responding validator is recorded on-chain and subject to slashing.

Definition 1 (Session Liveness)**.** A session $\mathcal{S}$ completes if at least t of the n participants submit valid round messages within every deadline. Formally, for each round $r \in \{1, \ldots, R\}$:

$$|\{i \in \mathcal{S}.\texttt{participants} : \texttt{msg}_{i,r} \text{ valid}\}| \geq t$$

2.4 Permissioned Key Creation

Not every actor may initiate a keygen. T-Chain enforces an `AuthorizedChains` registry: only chains listed in this registry (e.g., the EVM L2 bridge contract, the Identity Chain) may request key creation. This prevents spam and ensures that every generated key serves a concrete cross-chain or custodial purpose.

3 FROST Protocol

3.1 Overview

FROST (Flexible Round-Optimized Schnorr Threshold signatures) [1] implements t-of-n Schnorr signing in two rounds, improving on prior schemes that required $O(n)$ rounds. T-Chain uses FROST for all Ed25519 and Schnorr-compatible chains.

3.2 Distributed Key Generation

FROST DKG proceeds in two phases:

1. **Commitment phase**: Each participant P_i samples a random polynomial $f_i(x)$ of degree $t-1$, computes commitments $C_{i,j} = g^{a_{i,j}}$ for each coefficient $a_{i,j}$, and broadcasts $(C_{i,0}, \ldots, C_{i,t-1})$ together with a proof of knowledge of $a_{i,0}$.

2. **Share phase**: Each P_i sends the evaluation $f_i(j)$ to participant P_j over a confidential channel. P_j verifies consistency against the commitments:

$$g^{f_i(j)} = \prod_{k=0}^{t-1} C_{i,k}^{j^k}$$

The group public key is $Y = \prod_{i=1}^{n} C_{i,0}$ and each participant holds secret share $s_j = \sum_{i=1}^{n} f_i(j)$.

3.3 Two-Round Signing

Given a message m and signing set $S \subseteq \{1, \ldots, n\}$ with $|S| \geq t$:

1. **Round 1 (Nonce commitment)**: Each signer P_i samples nonces (d_i, e_i), computes commitments $(D_i = g^{d_i}, E_i = g^{e_i})$, and broadcasts (D_i, E_i).

2. **Round 2 (Partial signature)**: Compute binding factor $\rho_i = H(\texttt{msg\_hash}, i, D_i, E_i, B)$ where B is the full commitment list. Each signer produces:

$$z_i = d_i + \rho_i e_i + \lambda_i \cdot s_i \cdot c$$

where $c = H(R, Y, m)$, $R = \prod_{i \in S}(D_i \cdot E_i^{\rho_i})$, and λ_i is the Lagrange coefficient for participant i in set S.

The aggregated signature is (R, z) where $z = \sum_{i \in S} z_i$, verifiable as a standard Schnorr signature under Y.

Theorem 1 (t-of-n Unforgeability)**.** *Under the Discrete Logarithm assumption in the random oracle model, no PPT adversary controlling fewer than t participants can produce a valid* FROST *signature on a message not signed by an honest quorum.*

4 CGGMP21 Protocol

4.1 Overview

CGGMP21 [2] implements threshold ECDSA without ever reconstructing the secret key, making it suitable for Bitcoin, Ethereum, and all secp256k1/secp256r1 chains. T-Chain uses CGGMP21 for all ECDSA-based signing.

4.2 Key Properties

- **No key reconstruction**: At no point during keygen or signing is the full private key assembled in any single location.
- **Identifiable abort**: If any party deviates from the protocol, the honest parties can identify the cheater and abort with proof of misbehavior.
- **Presignature pipeline**: Signing is split into an expensive offline phase (presignature generation) and a fast online phase (message-dependent completion).

4.3 Offline/Online Separation

Signing is split into an expensive *offline* phase and a fast *online* phase. The offline phase produces a presignature tuple (R, k_i, χ_i) independent of the message: each party samples $k_i, \gamma_i \in \mathbb{Z}_q$, broadcasts Paillier-encrypted $K_i = \text{Enc}(k_i)$ with range proofs, and runs multiplicative-to-additive (MtA) subprotocols to compute shares δ_i such that $\sum \delta_i = k \cdot \gamma \pmod q$. From $\delta = k\gamma$, parties recover $R = g^{k^{-1}}$ and compute χ_i (shares of $k \cdot x$).

In the online phase, given message hash $H(m)$ and a presignature, each party computes $\sigma_i = k_i \cdot H(m) + r \cdot \chi_i \pmod q$ where r is the x-coordinate of R. The aggregator sums $\sigma = \sum_{i \in S} \sigma_i$ and outputs the ECDSA signature (r, σ). The online phase requires a single round, enabling sub-second latency.

Theorem 2 (CGGMP21 Security)**.** *Under the Strong RSA assumption, the DDH assumption, and in the random oracle model, the* CGGMP21 *protocol achieves t-of-n unforgeability with identifiable abort: any deviating party is identified with probability 1.*

5 LSS Foundation

5.1 Linear Secret Sharing

All threshold protocols in T-Chain rest on a common Linear Secret Sharing (LSS) layer. A (t, n)-LSS scheme distributes a secret s into shares $(s_1, \ldots, s_n)$ via a degree-$(t-1)$ polynomial over $\mathbb{Z}_q$:

$$f(x) = s + a_1 x + a_2 x^2 + \cdots + a_{t-1} x^{t-1}, \quad s_i = f(i) \tag{1}$$

Any t shares reconstruct s via Lagrange interpolation; fewer than t shares reveal no information about s (information-theoretic secrecy).

5.2 The Composition Theorem

The key insight enabling T-Chain's multi-protocol architecture is that protocols built on compatible LSS instances can share the same access structure and secret shares.

Theorem 3 (LSS Composition)**.** *Let $\Pi_1, \ldots, \Pi_m$ be threshold cryptographic protocols, each requiring a* (t, n)-LSS *over the same finite field $\mathbb{F}_q$. If each Π_j uses shares of the form $s_i^{(j)} = f^{(j)}(i)$ for independent polynomials $f^{(j)}$ but the same evaluation points $\{1, \ldots, n\}$, then a single keygen ceremony can produce correlated shares for all protocols simultaneously via a vector polynomial:*

$$\mathbf{f}(x) = (f^{(1)}(x), \ldots, f^{(m)}(x))$$

The resulting combined scheme preserves the t-privacy and t-correctness guarantees of each individual protocol.

This theorem is formally verified in `Crypto/Threshold/Composition.lean` and underpins the following protocol combinations in T-Chain:

- **FROST**: Schnorr threshold signatures (Ed25519, Ristretto)
- **CMP** (CGGMP21): ECDSA threshold signatures (secp256k1, secp256r1)
- **TFHE**: Threshold fully-homomorphic encryption decryption
- **Ringtail**: Ring signature scheme with threshold key generation
- **BLS**: BLS threshold signatures for beacon chain compatibility

All five protocols derive their shares from a single DKG ceremony, reducing validator coordination overhead from $O(m)$ independent DKGs to a single vectorized DKG.

6 MPC Wallet Creation

6.1 Permissioned Keygen

Wallet creation on T-Chain is gated by the `AuthorizedChains` registry. A requesting chain (e.g., the Zoo EVM L2 bridge) submits a `KeygenTx` specifying:

- `chain_id`: The requesting chain's identifier.
- `threshold`: The desired t value.
- `key_type`: `schnorr` (FROST) or `ecdsa` (CGGMP21).
- `metadata`: Application-specific context (e.g., bridge address, DID document hash).

6.2 Keygen Flow

1. The `KeygenTx` enters the ThresholdVM mempool and is included in a block.
2. ThresholdVM selects n validators from the active set, weighted by stake.
3. A DKG session $\mathcal{S}$ is created; each selected validator receives a round-1 prompt.
4. Validators execute the DKG (FROST or CGGMP21) across R rounds, posting round messages as chain transactions.
5. On successful completion, ThresholdVM records the group public key Y and `key_id` in the chain state.
6. Each validator stores their secret share s_i in local encrypted storage, keyed by `key_id`.

6.3 Public Key Derivation

For FROST keys, the group public key Y is a standard Ed25519 point. For CGGMP21 keys, Y is a secp256k1 point. In both cases, the corresponding blockchain address is derived using the target chain's standard address derivation (e.g., Keccak-256 for Ethereum, SHA-256 + RIPEMD-160 for Bitcoin). Hierarchical derivation follows BIP-32 for ECDSA keys, as described in [16].

7 Security Analysis

7.1 Open DKG Safety

The DKG executes over an open network where validators may be adversarial. Safety relies on two mechanisms:

1. **Feldman commitments**: Each participant commits to their polynomial coefficients. Share recipients verify consistency, detecting any deviation.
2. **Zero-knowledge proofs**: Each participant proves knowledge of their secret coefficient $a_{i,0}$ without revealing it, preventing a rogue-key attack where an adversary biases the group public key.

Proposition 4 (DKG Soundness)**.** *If fewer than t of the n DKG participants are corrupted, the resulting group public key Y is uniformly distributed and the adversary learns no information about honest parties' shares.*

7.2 Consensus-Coupled Threshold Guarantees

In T-Chain, the MPC threshold t is set to match the consensus fault tolerance. For n validators with total stake W, the consensus safety threshold requires honest control of $\geq \frac{2}{3}W$. We set $t = \lceil \frac{n}{3} \rceil + 1$, ensuring:

$$\text{Forge signature} \implies \text{Corrupt } \geq t \text{ validators} \implies \text{Break consensus} \tag{2}$$

An adversary who can forge a threshold signature can also break consensus, and vice versa. The two security boundaries collapse into one.

7.3 Identifiable Abort

Both FROST and CGGMP21 support identifiable abort. If a participant submits an invalid round message:

1. Other participants detect the inconsistency via commitment verification or range proof checking.
2. The session aborts and the offending validator's identity is recorded on-chain.
3. The validator is slashed according to the staking parameters defined in [13].
4. A new session is initiated excluding the slashed validator.

This mechanism converts MPC liveness failures into economic penalties, aligning cryptographic and economic security.

8 GPU Acceleration

8.1 FHE Operations in T-Chain

Threshold FHE decryption is the most computationally intensive operation in T-Chain. Each validator holds a share of the FHE decryption key and must perform partial decryption on ciphertexts that may encode vectors of thousands of elements. T-Chain offloads these operations to GPU:

- **NTT (Number Theoretic Transform)**: Polynomial multiplication in RLWE-based FHE uses NTT, which parallelizes across GPU cores. We observe 47× speedup over CPU for degree-2^{16} polynomials.
- **Batch partial decryption**: Multiple ciphertexts are decrypted in a single GPU kernel launch, amortizing launch overhead.
- **ZK proof generation**: Range proofs and commitment verification for CGGMP21 presignatures leverage GPU-parallel multi-scalar multiplication (MSM).

8.2 Performance

Operation	CPU (ms)	GPU (ms)	Speedup
FROST 2-round sign (t=10, n=15)	12	3	4×
CGGMP21 presignature (t=10, n=15)	840	58	14.5×
TFHE partial decrypt (degree 2^{16})	2,350	50	47×
ZK range proof (Bulletproofs)	180	22	8.2×

Table 2: GPU acceleration benchmarks (NVIDIA A100, single GPU)

These benchmarks demonstrate that GPU acceleration makes threshold FHE decryption practical at consensus timescales. See [14] for the full FHE architecture.

9 Formal Verification

All threshold protocols in T-Chain are formally verified in Lean 4 as part of the Zoo Crypto library:

- `Crypto/FROST.lean`: Correctness of FROST DKG and 2-round signing. The key theorem states that honest aggregation of t valid partial signatures yields a signature verifiable under the group public key.
- `Crypto/Threshold/CGGMP21.lean`: Correctness of presignature generation and online signing. Identifiable abort is proved: if any party deviates, at least one honest party can produce a proof of misbehavior.
- `Crypto/Threshold/LSS.lean`: t-privacy (fewer than t shares reveal nothing) and t-correctness (any t shares reconstruct the secret) for Shamir's scheme over arbitrary finite fields.
- `Crypto/Threshold/Composition.lean`: The Composition Theorem (Theorem 3), proving that vectorized DKG preserves individual protocol guarantees.

Remark 1. The Lean proofs cover functional correctness and information-theoretic secrecy. Computational hardness assumptions (DL, Strong RSA, DDH) are axiomatized as hypotheses; the proofs show that security reduces to these assumptions.

10 Migration from External MPC

10.1 Replacing the MPC StatefulSet

Prior to T-Chain, Zoo Network's bridge and custody operations ran on a dedicated MPC cluster deployed as a Kubernetes StatefulSet (`mpc-signer-0` through `mpc-signer-n`). This architecture suffered from:

- Manual key rotation requiring coordinated downtime.
- Separate monitoring and alerting stacks.
- No on-chain accountability for MPC failures.
- Independent scaling from the validator set.

T-Chain subsumes all functionality of the MPC StatefulSet. The migration proceeds in three phases:

1. **Phase 1 – Parallel operation**: T-Chain runs alongside the legacy MPC cluster. New keys are generated on T-Chain; existing keys remain on the StatefulSet.
2. **Phase 2 – Key migration**: Existing keys are reshared from the StatefulSet participants to the T-Chain validator set via the `threshold_reshare` protocol.
3. **Phase 3 – Decommission**: The MPC StatefulSet is drained and removed. All signing requests route to T-Chain.

10.2 Bridge Integration

Zoo's cross-chain bridges (connecting to Lux L0, Ethereum, Bitcoin) previously called the MPC cluster via internal gRPC. T-Chain exposes an HTTP compatibility layer at `/v1/threshold/compat` that accepts the legacy request format and translates it to ThresholdVM transactions. This allows bridge contracts to migrate incrementally without a flag-day cutover.

10.3 HTTP Compatibility Layer

The compatibility layer translates legacy MPC API calls:

Legacy Endpoint	T-Chain Translation
`POST /mpc/keygen`	`threshold_keygen`
`POST /mpc/sign`	`threshold_sign`
`POST /mpc/reshare`	`threshold_reshare`
`GET /mpc/key/:id`	`threshold_pubkey`
`GET /mpc/status/:session`	`threshold_status`

Table 3: Legacy MPC to T-Chain endpoint mapping

11 Related Work

Lux threshold infrastructure. Lux Network provides the foundational libraries used by T-Chain. The `lux-universal-threshold-signatures` library [17] implements FROST and CGGMP21 at the primitive level; `lux-mchain-mpc` [18] pioneered multi-chain MPC on Lux but operated as an external service. T-Chain embeds these primitives directly into a VM.

Fireblocks MPC. Fireblocks [8] offers institutional MPC-CMP as a managed service. While production-proven, it introduces a trusted third party. T-Chain achieves comparable guarantees without custodial trust assumptions.

Lit Protocol. Lit [9] distributes threshold signing across a dedicated node network, requiring separate infrastructure. T-Chain reuses the validator set, reducing cost and aligning incentives.

THORChain TSS. THORChain [10] uses GG20 for cross-chain swaps. T-Chain improves on this with identifiable abort (absent in GG20) and the Composition Theorem for multi-protocol share reuse.

DFINITY threshold ECDSA. The Internet Computer [11] implements chain-key threshold ECDSA at the subnet level. T-Chain follows a similar chain-native philosophy but extends it to FROST, TFHE, and BLS via the LSS Composition Theorem.

12 Conclusion

T-Chain demonstrates that threshold signature protocols belong inside the consensus layer, not beside it. By making validators the MPC parties, Zoo Network achieves:

1. **Unified security**: The MPC quorum and the consensus quorum are identical. No separate trust boundary exists for custody.
2. **Operational simplicity**: No sidecar deployments, no independent key rotation schedules, no separate monitoring.
3. **Economic alignment**: MPC failures trigger the same slashing penalties as consensus failures, eliminating free-rider incentives.
4. **Protocol composability**: The LSS Composition Theorem enables a single DKG ceremony to provision shares for FROST, CGGMP21, TFHE, Ringtail, and BLS simultaneously.
5. **Verified correctness**: All protocols are formally verified in Lean 4, with computational security reducing to standard hardness assumptions.

The migration path from external MPC to T-Chain is incremental: parallel operation, key reshare, decommission. Existing bridge integrations continue to function through the HTTP compatibility layer. GPU acceleration ensures that even computationally heavy operations (threshold FHE decryption) complete within consensus block times.

T-Chain establishes the foundation for chain-native cryptographic services on Zoo Network, enabling future extensions including threshold decryption for privacy-preserving inference [14], DID-bound threshold keys for decentralized identity [15], and cross-chain atomic swaps without trusted bridges.

References

[1] Chelsea Komlo and Ian Goldberg. *FROST: Flexible Round-Optimized Schnorr Threshold Signatures*. Selected Areas in Cryptography (SAC), 2020.

[2] Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. *UC Non-Interactive, Proactive, Threshold ECDSA with Identifiable Aborts*. ACM CCS, 2020.

[3] Adi Shamir. *How to Share a Secret*. Communications of the ACM, 22(11):612–613, 1979.

[4] Paul Feldman. *A Practical Scheme for Non-Interactive Verifiable Secret Sharing*. FOCS, 1987.

[5] Torben Pryds Pedersen. *Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing*. CRYPTO, 1991.

[6] Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. *Secure Distributed Key Generation for Discrete-Log Based Cryptosystems*. Journal of Cryptology, 20(1):51–83, 2007.

[7] Yehuda Lindell. *Fast Secure Two-Party ECDSA Signing*. CRYPTO, 2017.

[8] Fireblocks. *MPC-CMP: Next Generation MPC for Digital Asset Security*. Fireblocks Technical Paper, 2023.

[9] Lit Protocol. *Threshold Cryptography for Decentralized Access Control*. Lit Protocol Whitepaper, 2023.

[10] THORChain. *TSS: Threshold Signature Scheme Implementation*. THORChain Documentation, 2023.

[11] DFINITY Foundation. *Chain-Key Threshold ECDSA on the Internet Computer*. DFINITY Technical Report, 2022.

[12] Zach Kelling. *Zoo Network Architecture: A Multi-Chain AI Infrastructure*. Zoo Labs Foundation, 2025.

[13] Zach Kelling. *Zoo Network Tokenomics: Fair Distribution Through Complete Airdrop*. Zoo Labs Foundation, 2025.

[14] Zach Kelling. *Fully Homomorphic Encryption on Zoo Network*. Zoo Labs Foundation, 2026.

[15] Zach Kelling. *Zoo Identity Chain: Decentralized Identity and DID Management*. Zoo Labs Foundation, 2026.

[16] Zach Kelling. *Zoo Hierarchical Key Management*. Zoo Labs Foundation, 2026.

[17] Lux Partners. *Universal Threshold Signatures for Multi-Consensus Networks*. Lux Technical Report, 2025.

[18] Lux Partners. *Multi-Chain MPC Signing on Lux Network*. Lux Technical Report, 2025.